

Natural Evolution Strategies (NES)

Chiến lược Tiến hóa Tự nhiên

Lê Thiên Quân - 24520027
Đinh Mạnh Hùng - 24520014

Khoa Khoa học Máy tính
Trường Đại học Công nghệ Thông tin - ĐHQG TP.HCM

Ngày 28 tháng 12 năm 2025



Nội dung trình bày

- 1 Chương 1: Giới thiệu & Bối cảnh
- 2 Chương 2: Cơ sở Toán học của NES
- 3 Chương 3: Các phiên bản "Đỉnh cao" của họ NES
- 4 Chương 4: So sánh & Kết luận



Chương 1:

Giới thiệu & Bối cảnh



1.1. Bài toán Tối ưu hóa Hộp đen (Black-Box Optimization)

Thực trạng: Nhiều bài toán thực tế (Y tế, Hàng không, Điều khiển tự động...) quá phức tạp để mô hình hóa trực tiếp.

Đặc điểm "Hộp đen":

- Không có thông tin về cấu trúc hàm mục tiêu $f(x)$.
- Không có đạo hàm giải tích.
- Chỉ có thể đánh giá $f(x)$ tại các điểm tham số cụ thể.

Mục tiêu

Tìm cực trị toàn cục mà không cần hiểu "bên trong" hàm số hoạt động như thế nào, thường dựa trên các phương pháp heuristic.



1.2. Lược sử các phương pháp tiếp cận

Bài toán này đã được nghiên cứu hơn 50 năm qua với nhiều hướng tiếp cận:

- **Cổ điển:** Nelder-Mead (Simplex), Quasi-Newton.
- **Lấy cảm hứng sinh học/vật lý:**
 - Simulated Annealing (Luyện kim).
 - Genetic Algorithms (Di truyền học).
 - Particle Swarm Optimization (Bầy đàn).
- **Evolution Strategies (ES):** Phát triển bởi Rechenberg & Schwefel (thập niên 60-70). Dùng đột biến (mutation) và chọn lọc (selection) trên các số thực.



1.3. Từ Heuristics đến Toán học chặt chẽ

CMA-ES (Covariance Matrix Adaptation - ES):

- Là chuẩn mực vàng của dòng ES.
- Sử dụng ma trận hiệp phương sai để mô hình hóa sự phụ thuộc giữa các biến.
- *Tuy nhiên*: Dựa nhiều vào heuristics phức tạp, khó phân tích động lực học một cách chính xác.

Yêu cầu: Cần một phương pháp có hiệu năng tương đương CMA-ES nhưng có *nền tảng toán học minh bạch* thay vì các quy tắc tay (hand-crafted rules). → **Sự ra đời của Natural Evolution Strategies (NES).**



1.4. Ý tưởng cốt lõi của NES

Khác với các phương pháp ES truyền thống (thường chọn lọc cá thể rồi tạo quần thể mới), NES tiếp cận theo hướng **Gradient Ascent trên Phân phối**:

Cơ chế hoạt động:

- 1 Duy trì một phân phối tìm kiếm $\pi(z|\theta)$ (VD: Gaussian).
- 2 Cập nhật tham số θ (VD: μ, Σ) theo hướng tăng kỳ vọng hàm thích nghi.
- 3 Sử dụng **Natural Gradient** để đảm bảo sự ổn định.

Sự khác biệt

NES không tối ưu hóa điểm dữ liệu. NES tối ưu hóa **phân phối** sinh ra dữ liệu đó.



1.5. Tiếp

Để thực hiện ý tưởng trên, chúng ta cần trả lời 2 câu hỏi lớn:

- ❶ Làm thế nào để tính gradient của một "kỳ vọng" khi không biết hàm $f(x)$?
- ❷ Tại sao Gradient thông thường lại thất bại trong việc cập nhật tham số phân phối (đặc biệt là phương sai)?

Chương 2 sẽ đi sâu vào cơ sở lý thuyết giải quyết hai vấn đề này.



Chương 2:

Search Gradients & Natural Gradient



2.1. Định nghĩa Search Gradient

Để giải quyết bài toán hộp đen một cách toán học, NES tối ưu hóa **Kỳ vọng hàm thích nghi** (Expected Fitness) thay vì tối ưu trực tiếp $f(x)$.

Gọi θ là tham số của phân phối tìm kiếm $\pi(z|\theta)$. Hàm mục tiêu $J(\theta)$ được định nghĩa (Eq. 1):

$$J(\theta) = \mathbb{E}_{\theta}[f(z)] = \int f(z)\pi(z|\theta)dz \quad (1)$$

Vấn đề: Chúng ta cần tìm $\nabla_{\theta}J(\theta)$ để thực hiện Gradient Ascent:

$$\theta \leftarrow \theta + \eta \nabla_{\theta}J(\theta)$$

Nhưng $f(z)$ không khả vi hoặc không biết công thức. Làm sao tính đạo hàm?



2.2. Log-Likelihood Trick

Đây là bước quan trọng nhất để chuyển đạo hàm vào trong phân phối xác suất.

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \nabla_{\theta} \int f(z) \pi(z|\theta) dz \\&= \int f(z) \nabla_{\theta} \pi(z|\theta) dz \quad (\text{Đưa đạo hàm vào trong tích phân}) \\&= \int f(z) \frac{\nabla_{\theta} \pi(z|\theta)}{\pi(z|\theta)} \pi(z|\theta) dz \quad (\text{Nhân và chia cho } \pi) \\&= \int f(z) \nabla_{\theta} \log \pi(z|\theta) \pi(z|\theta) dz \quad (\text{Quy tắc đạo hàm log}) \\&= \mathbb{E}_{z \sim \pi} [f(z) \nabla_{\theta} \log \pi(z|\theta)]\end{aligned}\tag{2}$$

Kết quả

Ta chỉ cần tính đạo hàm của mật độ xác suất $\log \pi$ (thứ ta đã biết), không cần đạo hàm của $f(z)$ (thứ ta không biết).



2.3. Ước lượng Monte Carlo

Trong thực tế, ta không tính tích phân mà xấp xỉ bằng λ mẫu thử $z_1, \dots, z_\lambda$ được sinh từ $\pi(\cdot|\theta)$:

$$\nabla_\theta J(\theta) \approx \frac{1}{\lambda} \sum_{k=1}^{\lambda} f(z_k) \nabla_\theta \log \pi(z_k|\theta) \quad (3)$$

Đây chính là nền tảng của thuật toán **Canonical Search Gradient**.

- $\nabla_\theta \log \pi(z|\theta)$: Gọi là *Score Function* (cho biết hướng thay đổi tham số để tăng xác suất sinh ra mẫu z).
- $f(z_k)$: Đóng vai trò trọng số.



2.4. Áp dụng cho Phân phối Gaussian đa biến

Xét trường hợp phổ biến nhất: Phân phối chuẩn d -chiều $\mathcal{N}(\mu, \Sigma)$.

- Tham số $\theta = \langle \mu, \Sigma \rangle$.
- Log-density:

$$\log \pi(z|\theta) = -\frac{d}{2} \log(2\pi) - \frac{1}{2} \log \det \Sigma - \frac{1}{2} (z - \mu)^\top \Sigma^{-1} (z - \mu)$$

Đạo hàm riêng (Search Gradients) cụ thể như sau:

$$\nabla_{\mu} \log \pi(z) = \Sigma^{-1} (z - \mu) \quad (4)$$

$$\nabla_{\Sigma} \log \pi(z) = \frac{1}{2} \Sigma^{-1} (z - \mu) (z - \mu)^\top \Sigma^{-1} - \frac{1}{2} \Sigma^{-1} \quad (5)$$



2.5. Tại sao Search Gradient thông thường thất bại?

Xét trường hợp 1 chiều đơn giản với $\theta = (\mu, \sigma)$. Gradient đối với σ là:

$$\nabla_{\sigma} J \propto \frac{(z - \mu)^2 - \sigma^2}{\sigma^3}$$

Sự mất ổn định:

- Khi thuật toán cần hội tụ chính xác, σ phải giảm dần ($\sigma \rightarrow 0$).
- Tuy nhiên, độ lớn bước nhảy (step size) lại tỉ lệ nghịch với σ^3 .
- **Kết quả:** Khi σ nhỏ, bước cập nhật trở nên khổng lồ ("Overshooting"). Thuật toán bị văng ra xa khỏi cực trị hoặc dao động mạnh.

Kết luận: Gradient Euclidean không "Scale-invariant" (Bất biến theo tỷ lệ).



2.6. Giải pháp: Natural Gradient

Thay vì dùng khoảng cách Euclidean trong không gian tham số, ta dùng khoảng cách giữa các phân phối xác suất (Kullback-Leibler Divergence).

Bài toán tối ưu ràng buộc: Tìm bước nhảy $\delta\theta$ sao cho tối đa hóa hàm mục tiêu, nhưng giới hạn sự thay đổi của phân phối:

$$\begin{aligned} \max_{\delta\theta} \quad & J(\theta + \delta\theta) \approx J(\theta) + \delta\theta^\top \nabla_\theta J \\ \text{s.t.} \quad & D_{KL}(\pi(\cdot|\theta + \delta\theta) || \pi(\cdot|\theta)) = \varepsilon \end{aligned} \tag{6}$$

Trong đó $D_{KL} \approx \frac{1}{2} \delta\theta^\top F(\theta) \delta\theta$ (xấp xỉ Taylor bậc 2).



2.7. Ma trận Thông tin Fisher (F)

Giải bài toán tối ưu trên bằng nhân tử Lagrange, ta thu được hướng cập nhật tối ưu (Natural Gradient):

$$\tilde{\nabla}_{\theta} J = F^{-1} \nabla_{\theta} J \quad (7)$$

Với F là ma trận Fisher, được định nghĩa là hiệp phương sai của score function:

$$F = \mathbb{E} [\nabla_{\theta} \log \pi(z|\theta) \nabla_{\theta} \log \pi(z|\theta)^{\top}] \quad (8)$$

Ý nghĩa của F^{-1} :

- Nó đóng vai trò như một bộ "tiền điều kiện" (Pre-conditioner).
- Nó triệt tiêu ảnh hưởng của độ cong không gian tham số (ví dụ: nhân tử $1/\sigma^3$ sẽ bị F triệt tiêu), giúp bước cập nhật ổn định bất kể σ lớn hay nhỏ.



2.8. Tổng hợp: Thuật toán Canonical NES

Algorithm 1 Canonical Natural Evolution Strategies

```
1: Input:  $f, \theta_{init}$ , learning rate  $\eta$ 
2: repeat
3:   for  $k = 1 \dots \lambda$  do
4:     Draw  $z_k \sim \pi(\cdot|\theta)$ 
5:     Evaluate  $f(z_k)$ 
6:     Calculate gradients  $\nabla_{\theta} \log \pi(z_k|\theta)$ 
7:   end for
8:    $\nabla_{\theta} J \leftarrow \frac{1}{\lambda} \sum_{k=1}^{\lambda} f(z_k) \nabla_{\theta} \log \pi(z_k|\theta)$ 
9:    $F \leftarrow \frac{1}{\lambda} \sum_{k=1}^{\lambda} \nabla_{\theta} \log \pi(z_k|\theta) \nabla_{\theta} \log \pi(z_k|\theta)^{\top}$ 
10:   $\theta \leftarrow \theta + \eta F^{-1} \nabla_{\theta} J$  {Natural Update Step}
11: until stopping criterion met
```



Chương 3:

Các phiên bản "Đỉnh cao" của họ NES



3.1. xNES (Exponential NES)

Đặc trưng: Sử dụng ma trận hiệp phương sai đầy đủ (*Full Covariance*), cho phép học mối tương quan giữa các chiều.

Quy trình chi tiết (xNES Pipeline):

- ➊ **Khởi tạo:** Tâm μ và ma trận hiệp phương sai thông qua σ và B (với $A = \sigma B$ và B có $\det(B) = 1$).
- ➋ **Sinh mẫu chuẩn:** Lấy λ mẫu $s_k \sim \mathcal{N}(0, I)$.
- ➌ **Biến đổi mẫu:** Tạo các cá thể $z_k = \mu + \sigma B s_k$.
- ➍ **Đánh giá:** Tính $f(z_k)$ và quy đổi sang Utilities u_k dựa trên hạng.
- ➎ **Tính Gradient cục bộ:** Tính đạo hàm cho μ, σ và ma trận B trong hệ tọa độ tự nhiên (nơi $F = I$).
- ➏ **Cập nhật kiểu nhân (Exponential Update):**
 - $\mu \leftarrow \mu + \eta_\mu \sigma B \nabla_\delta$
 - $\sigma \leftarrow \sigma \cdot \exp(\frac{\eta_\sigma}{2} \nabla_\sigma)$
 - $B \leftarrow B \cdot \exp(\eta_B \nabla_B)$



3.2. Thuật toán xNES (Pseudo-code)

Algorithm 2 Exponential NES (xNES)

- 1: Sample $s_k \sim \mathcal{N}(0, I)$ for $k = 1 \dots \lambda$
 - 2: $z_k \leftarrow \mu + \sigma B s_k$
 - 3: Compute gradients for local coordinates:
 - 4: $\nabla_\delta \leftarrow \sum u_k s_k, \quad \nabla_\sigma \leftarrow \sum u_k (\|s_k\|^2 - d)/d$
 - 5: $\nabla_B \leftarrow \sum u_k (s_k s_k^T - I - \frac{\|s_k\|^2 - d}{d} I)$
 - 6: $\mu \leftarrow \mu + \eta_\mu \sigma B \nabla_\delta$
 - 7: $\sigma \leftarrow \sigma \cdot \exp(\frac{\eta_\sigma}{2} \nabla_\sigma)$
 - 8: $B \leftarrow B \cdot \exp(\eta_B \nabla_B)$ {Matrix exponential}
-



3.3. SNES (Separable NES)

Đặc trưng: Giả định các biến độc lập (*Diagonal Covariance*). Phù hợp cho bài toán rất lớn (high-dimensional). **Quy trình chi tiết (SNES Pipeline):**

- ❶ **Khởi tạo:** Vector trung bình μ và vector độ lệch chuẩn σ (cùng số chiều d).
- ❷ **Sinh mẫu:** Lấy λ mẫu $s_k \sim \mathcal{N}(0, I)$.
- ❸ **Biến đổi mẫu:** $z_{k,i} = \mu_i + \sigma_i s_{k,i}$ (tính toán độc lập trên từng chiều i).
- ❹ **Đánh giá:** Tính $f(z_k)$ và quy đổi sang Utilities u_k .
- ❺ **Tính Gradient thành phần:**
 - $\nabla_{\mu,i} = \sum u_k s_{k,i}$
 - $\nabla_{\sigma,i} = \sum u_k (s_{k,i}^2 - 1)$
- ❻ **Cập nhật độc lập:**
 - $\mu_i \leftarrow \mu_i + \eta_{\mu} \sigma_i \nabla_{\mu,i}$
 - $\sigma_i \leftarrow \sigma_i \cdot \exp(\frac{\eta_{\sigma}}{2} \nabla_{\sigma,i})$



3.4. Thuật toán SNES (Pseudo-code)

Algorithm 3 Separable NES (SNES)

- 1: Sample $s_k \sim \mathcal{N}(0, I)$ for $k = 1 \dots \lambda$
 - 2: $z_{k,i} \leftarrow \mu_i + \sigma_i s_{k,i}$ for each dimension i
 - 3: Compute element-wise gradients:
 - 4: $\nabla_{\mu,i} \leftarrow \sum_{k=1}^{\lambda} u_k s_{k,i}$
 - 5: $\nabla_{\sigma,i} \leftarrow \sum_{k=1}^{\lambda} u_k (s_{k,i}^2 - 1)$
 - 6: $\mu_i \leftarrow \mu_i + \eta_{\mu} \sigma_i \nabla_{\mu,i}$
 - 7: $\sigma_i \leftarrow \sigma_i \cdot \exp(\frac{\eta_{\sigma}}{2} \nabla_{\sigma,i})$
-



3.5. So sánh xNES vs SNES

Đặc điểm	xNES (Exponential)	SNES (Separable)
Cấu trúc Σ	Ma trận đầy đủ	Ma trận đường chéo
Độ phức tạp	$O(d^3)$	$O(d)$
Học tương quan	Có (Rất tốt)	Không
Khả năng mở rộng	Bài toán vừa ($d < 500$)	Bài toán lớn ($d > 10^6$)
Tính bất biến	Bất biến với phép quay	Không bất biến với phép quay



Chương 4:

So sánh & Thực nghiệm



4.1. NES vs CMA-ES

Cả hai đều là tiêu chuẩn vàng trong tối ưu hóa tiến hóa Gaussian.

- **Giống nhau:** Dùng phân phối Gaussian, dùng rank-based weights.
- **Khác nhau:**
 - **CMA-ES:** Dựa trên Heuristics (Evolution paths). Cực kỳ mạnh trong thực tế nhưng thiếu nền tảng toán học thống nhất.
 - **NES:** Dựa trên Natural Gradient (Fisher). Có cơ sở lý thuyết vững chắc, dễ mở rộng sang các phân phối khác.

Kết luận

CMA-ES là lựa chọn thực dụng; NES là lựa chọn cho nghiên cứu sâu và các bài toán cần tính bất biến tuyệt đối.



4.2. Thiết lập thực nghiệm

Bài toán: Tối ưu hóa hàm **Rastrigin** với $d = 2$.

$$f(\mathbf{x}) = 10d + \sum_{i=1}^d [x_i^2 - 10 \cos(2\pi x_i)]$$

- **Đặc điểm:** Non-convex, cực kỳ nhiều bẫy local minima.
- **Mục tiêu:** Tìm Global Minimum tại $\mathbf{x} = \mathbf{0}$, $f(\mathbf{x}) = 0$.

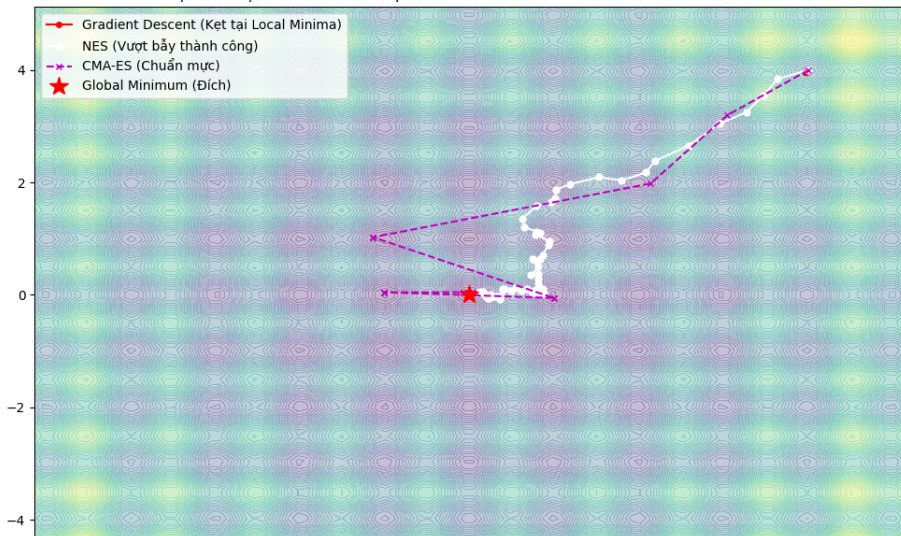
Cấu hình thuật toán:

- **NES:** Gaussian $\mathcal{N}(\mu, \sigma^2 I)$, $\lambda = 50$, Rank-based Utilities.
- **CMA-ES:** Cấu hình chuẩn (thư viện pycma).
- **Gradient Descent (GD):** Learning rate tối ưu, dùng analytic gradient.



4.3. Kết quả hội tụ (Convergence Analysis) với $d = 2$

THỰC NGHIỆM: KHẢ NĂNG VƯỢT LOCAL MINIMA CỦA NES TRÊN HÀM RASTRIGIN



4.3. Kết luận

- ① NES cung cấp nền tảng toán học vững chắc cho tiến hóa thông qua Hình học thông tin.
- ② Natural Gradient là chìa khóa để đạt được sự ổn định và tính bất biến.
- ③ xNES mang lại độ chính xác cao nhất, SNES mang lại hiệu suất cho các bài toán lớn.

Cảm ơn Thầy và các bạn!

